

|                        |                                                                                   |
|------------------------|-----------------------------------------------------------------------------------|
| EXPT NO: 5             | <b>IMPLEMENTATION OF A RECURRENT NEURAL NETWORK (RNN)<br/>FOR TEXT GENERATION</b> |
| DATE: 24th August,2026 |                                                                                   |

### **PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)**

#### **1. Explain the purpose of a Recurrent Neural Network (RNN) in processing sequential data.**

A Recurrent Neural Network (RNN) is designed to process sequential data by considering the order and relationship between elements. Unlike traditional neural networks, an RNN maintains a hidden state that carries information from previous time steps. This enables it to understand patterns and dependencies in sequences such as text, speech, time-series data, and sensor readings. Thus, RNNs are useful for tasks where past information influences the current output.

#### **2. Why is an RNN considered an appropriate model for text generation tasks?**

RNNs are appropriate for text generation because text is inherently sequential, where the meaning of a word often depends on previously occurring words. An RNN processes words one by one while maintaining information about earlier words through its hidden state. It can therefore learn language patterns, grammar, and word relationships. After training, the RNN can predict the next word based on the preceding sequence and generate meaningful text.

#### **3. Describe the concept of sequence modeling in natural language processing.**

Sequence modeling in Natural Language Processing (NLP) refers to analyzing and understanding data where the order of elements is important. In language, words appear in a specific sequence, and changing their order can change the meaning of a sentence. Sequence models learn relationships and dependencies between words or tokens. They are used for applications such as text generation, machine translation, sentiment analysis, speech recognition, and language prediction.

#### **4. Explain the importance of tokenization during text preprocessing for RNN-based text generation.**

Tokenization is an important preprocessing step that divides text into smaller units called tokens, such as words, subwords, or characters. RNNs cannot directly process raw text, so tokenization converts sentences into manageable sequences. Each token can then be assigned a numerical index for model processing. Proper tokenization helps the RNN identify meaningful patterns and relationships between words, improving its ability to learn language structure and generate text.

#### **5. Explain the significance of converting text into numerical representations before training an RNN.**

Converting text into numerical representations is essential because neural networks, including RNNs, operate on numerical data rather than raw characters or words. After tokenization, each token is mapped to a numerical value or vector representation. These numerical inputs allow the RNN to perform mathematical operations and learn patterns from the training data. Techniques such as word embeddings can also capture semantic relationships between words, improving text-generation performance.

## AIM

To implement a Recurrent Neural Network (RNN) for text generation and evaluate its ability to generate meaningful text by predicting the next word in a sequence.

## SOFTWARE REQUIREMENTS

- Python
- TensorFlow or keras or PyTorch
- Google Colab
- GitHub
- Numpy
- Matplotlib

## OBJECTIVES

Students will be able to:

- Understand the architecture and working of Recurrent Neural Networks (RNNs).
- Perform text preprocessing techniques for sequence modeling.
- Generate input sequences suitable for training an RNN model.
- Build and train an RNN model for text generation.
- Generate text by predicting the next word in a sequence.
- Evaluate the performance of the trained RNN using generated text.
- Analyze the effectiveness of RNNs in learning sequential patterns from textual data.

## PROCEDURE

### PART A: DATASET PREPARATION

Perform the following tasks:

- Load the **Tiny Shakespeare** dataset from Hugging Face.
- Explore the dataset and examine the text samples.
- Convert the text into lowercase (if required).
- Tokenize the text into individual words.
- Build a vocabulary and assign a unique integer index to each word.
- Generate input sequences for next-word prediction.
- Apply sequence padding to obtain inputs of equal length.
- Split the prepared sequences into training and validation datasets.

### PART B: IMPLEMENTING THE RNN MODEL

Perform the following tasks:

- Create an Embedding layer to convert words into dense vector representations.
- Add a Simple RNN layer to learn sequential dependencies in the text.
- Add a Dense output layer with Softmax activation for next-word prediction.
- Compile the model using an appropriate optimizer and loss function.
- Display the model architecture and summary.

### PART C: COMPARISON OF OPTIMIZATION ALGORITHMS

Perform the following tasks:

- Train the RNN model using the prepared training dataset.
- Validate the model using the validation dataset.
- Record:
  - Training accuracy
  - Validation accuracy
  - Training loss
  - Validation loss
- Visualize:
  - Accuracy vs. Epoch
  - Loss vs. Epoch

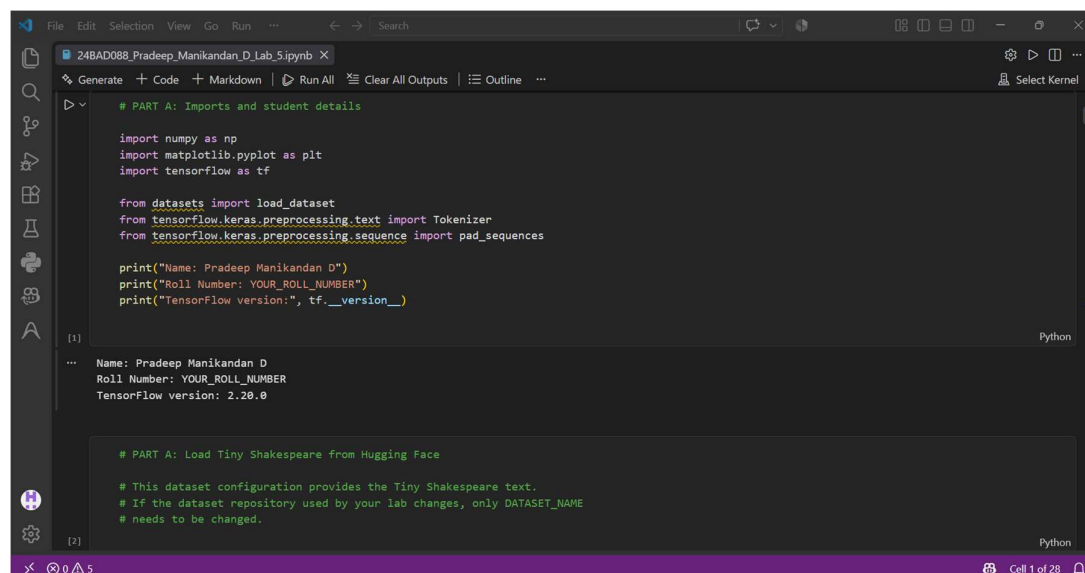
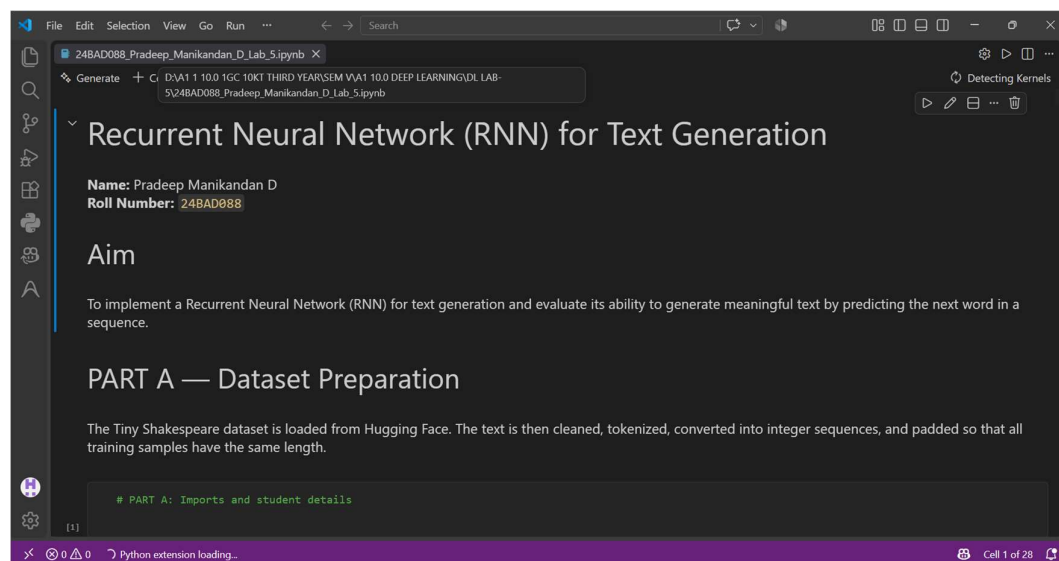
## PART D: DEEP LEARNING EXPERIMENT MANAGEMENT

Perform the following tasks:

- Provide a seed word or sentence as input to the trained model.
- Generate text by repeatedly predicting the next word in the sequence.
- Produce multiple text samples using different seed inputs.
- Compare the generated text for coherence and contextual relevance.
- Document the observations and upload the notebook to GitHub.

## EXPECTED OUTCOME

Students will understand how an RNN learns sequential information and generates meaningful text based on previously learned patterns. Students will learn to preprocess textual data and train a Recurrent Neural Network (RNN) for text generation. They will evaluate the model's performance using training metrics and generate text through next-word prediction.



```
File Edit Selection View Go Run ... Search
24BAD088_Pradeep_Manikandan_D_Lab_5.ipynb X
Generate + Code + Markdown Run All Clear All Outputs Outline ...
Select Kernel

DATASET_NAME = "Trelis/tiny-shakespeare"

dataset = load_dataset(DATASET_NAME)

print(dataset)
print("\nAvailable splits:", list(dataset.keys()))

[2] Python

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit

... README.md: 0% | 0.00/497 [00:00<?, ?B/s]
... train.csv: 0% | 0.00/1.22M [00:00<?, ?B/s]
... test.csv: 0% | 0.00/119k [00:00<?, ?B/s]
... Generating train split: 0% | 0/472 [00:00<?, ? examples/s]
... Generating test split: 0% | 0/49 [00:00<?, ? examples/s]
... DatasetDict({
  train: Dataset({
    features: ['Text'],
    num_rows: 472
  })
  test: Dataset({
```

```
File Edit Selection View Go Run ... Search
24BAD088_Pradeep_Manikandan_D_Lab_5.ipynb X
Generate + Code + Markdown Run All Clear All Outputs Outline ...
Select Kernel

})
test: Dataset({
  features: ['Text'],
  num_rows: 49
})
})

Available splits: ['train', 'test']

# Explore the dataset and identify the text column

split_name = list(dataset.keys())[0]
data = dataset[split_name]

print(data)
print("\nColumn names:", data.column_names)
print("\nFirst sample:")
print(data[0])

[3] Python

... Dataset({
  features: ['Text'],
  num_rows: 472
})

Column names: ['Text']
```

```
Column names: ['Text']

First sample:
{'Text': "First Citizen:\nBefore we proceed any further, hear me speak.\n\nAll:\nSpeak, speak.\n\nFirst Citizen:\nYou are all resolved rather to die t
```

```
# Extract text and display a sample
TEXT_COLUMN = "text"
if TEXT_COLUMN not in data.column_names:
    # Automatically find a likely text column if the expected name is unavailable.
    text_candidates = [c for c in data.column_names if data.features[c].dtype == "string"]
    if not text_candidates:
        raise ValueError("No text column was found in the selected dataset.")
    TEXT_COLUMN = text_candidates[0]
text = "\n".join(str(x) for x in data[TEXT_COLUMN])
print("Text column:", TEXT_COLUMN)
print("Number of characters:", len(text))
print("\nFirst 1000 characters:\n")
print(text[:1000])
```

```
[4]

Text column: Text
Number of characters: 1222825

First 1000 characters:
```

```
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.

All:
We know't, we know't.

First Citizen:
Let us kill him, and we'll have corn at our own price.
...
our pikes, ere we become rakes: for the gods know I
speak this in hunger for bread, not in thirst for revenge.
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

The image displays two screenshots of a Jupyter Notebook interface, likely from a web browser. The top screenshot shows a code cell with the following Python code:

```
# Convert text to lowercase

text = text.lower()

print(text[:500])
```

The output of this cell is a text snippet from a play, showing dialogue between characters. The bottom screenshot shows the same notebook with the code cell scrolled down to reveal more code:

```
# Tokenization and vocabulary creation

tokenizer = Tokenizer(
    filters='!"#$%&()*+,-./:;<=>@[\\]^_`{|}~\\t\\n',
    lower=True,
    oov_token=""
)

tokenizer.fit_on_texts([text])
```

The output of this cell is truncated, showing the beginning of the dialogue from the previous screenshot and then indicating that the output is truncated. The interface includes a sidebar with icons for file management, search, and other notebook functions. The top bar shows the file name "24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb" and various menu options like "Generate", "Code", "Markdown", "Run All", "Clear All Outputs", and "Outline".

```
File Edit Selection View Go Run ... Search
24BAD088_Pradeep_Manikandan_D_Lab_5.ipynb
Generate + Code + Markdown Run All Clear All Outputs Outline ... Select Kernel

total_words = len(tokenizer.word_index) + 1

print("Vocabulary size:", total_words)
print("\nFirst 20 vocabulary entries:")
print(list(tokenizer.word_index.items())[:20])

[6] Python

Vocabulary size: 12934

First 20 vocabulary entries:
[(' <OOV>', 1), ('o', 2), ('a', 3), ('i', 4), ('he', 5), ('d', 6), ('ha', 7), ('h', 8), ('e', 9), ('g', 10), ('\n', 11), ('of', 12), ('s', 13), ('his',

# Generate next-word prediction sequences

token_list = tokenizer.texts_to_sequences([text])[0]

# Keep the sequence length manageable for a classroom experiment.
SEQ_LENGTH = 10

input_sequences = []

for i in range(SEQ_LENGTH, len(token_list)):
    sequence = token_list[i-SEQ_LENGTH:i+1]
    input_sequences.append(sequence)

[7] Python

input_sequences = np.array(input_sequences)

X = input_sequences[:, :-1]
y = input_sequences[:, -1]

print("Input shape before padding:", X.shape)
print("Target shape:", y.shape)
print("Example input:", X[0])
print("Example target:", y[0])

[7] Python

Input shape before padding: (280417, 10)
Target shape: (280417,)
Example input: [ 297 173 222 1059 48 1536 3 31 614 21]
Example target: 70

# Apply sequence padding

X = pad_sequences(
    X,
    maxlen=SEQ_LENGTH,
    padding="pre",
    truncating="pre"

[8] Python
```

24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb

Generate + Code + Markdown Run All Select Kernel

Input shape after padding: (288417, 10)  
Example padded sequence: [ 297 173 222 1059 48 1536 3 31 614

```
# Train-validation split

split_index = int(0.9 * len(X))

X_train = X[:split_index]
y_train = y[:split_index]

X_val = X[split_index:]
y_val = y[split_index:]

print("Training samples:", len(X_train))
print("Validation samples:", len(X_val))
print("Training input shape:", X_train.shape)
print("Validation input shape:", X_val.shape)
```

[9] Python

Training samples: 252375  
Validation samples: 28842  
Training input shape: (252375, 10)  
Validation input shape: (28842, 10)

24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb

Generate + Code + Markdown Run All Select Kernel

PART B — Implementing the RNN Model

The model contains:

- An **Embedding** layer for dense word representations.
- A **SimpleRNN** layer for learning sequential dependencies.
- A **Dense + Softmax** output layer for next-word prediction.

```
# PART B: Build the Simple RNN model

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

EMBEDDING_DIM = 64
RNN_UNITS = 128

model = Sequential([
    Embedding(input_dim=total_words, output_dim=EMBEDDING_DIM, input_shape=(1, total_words)),
    SimpleRNN(RNN_UNITS),
    Dense(total_words, activation="softmax")
])
```

[10] Python

24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb

Generate + Code + Markdown Run All Select Kernel

```
model = Sequential([
    Embedding(input_dim=total_words, output_dim=EMBEDDING_DIM, input_shape=(1, total_words)),
    SimpleRNN(RNN_UNITS),
    Dense(total_words, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

[10] Python

```
Model: "sequential"
```

| Layer (type)           | Output Shape | Params        |
|------------------------|--------------|---------------|
| embedding (Embedding)  | ?            | 0 (untrained) |
| simple_rnn (SimpleRNN) | ?            | 0 (untrained) |
| dense (Dense)          | ?            | 0 (untrained) |

Total params: 0 (0.00 B)  
Trainable params: 0 (0.00 B)  
Non-trainable params: 0 (0.00 B)

Screenshot checkpoint — Part B: Capture the model summary showing the Embedding, SimpleRNN, and Dense layers.

## PART C — Training and Evaluation

The RNN is trained on the prepared sequences. Training and validation accuracy/loss are recorded and visualized against epochs.

```
# PART C: Train the RNN
EPOCHS = 10
BATCH_SIZE = 256

history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)
```

Epoch 1/10  
986/986 — 109s 109ms/step - accuracy: 0.0663 - loss: 3.7503  
Epoch 2/10  
986/986 — 138s 105ms/step - accuracy: 0.1434 - loss: 6.3996

```
# Record final training and validation metrics

final_train_accuracy = history.history["accuracy"][-1]
final_val_accuracy = history.history["val_accuracy"][-1]
final_train_loss = history.history["loss"][-1]
final_val_loss = history.history["val_loss"][-1]

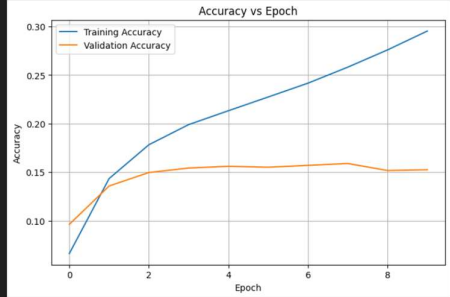
print(f"Training Accuracy : {final_train_accuracy:.4f}")
print(f"Validation Accuracy : {final_val_accuracy:.4f}")
print(f"Training Loss : {final_train_loss:.4f}")
print(f"Validation Loss : {final_val_loss:.4f}")
```

Training Accuracy : 0.2953  
Validation Accuracy : 0.1525  
Training Loss : 3.7503  
Validation Loss : 6.3996

```
# Accuracy vs Epoch

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Accuracy vs Epoch")
plt.show()
```

Accuracy vs Epoch



| Epoch | Training Accuracy | Validation Accuracy |
|-------|-------------------|---------------------|
| 1     | 0.0663            | 0.1434              |
| 2     | 0.1434            | 0.1525              |
| 3     | 0.2953            | 0.1525              |

```
Training Accuracy : 0.2953
Validation Accuracy : 0.1525
Training Loss : 3.7503
Validation Loss : 6.3996
```

```
# Accuracy vs Epoch

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Accuracy vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

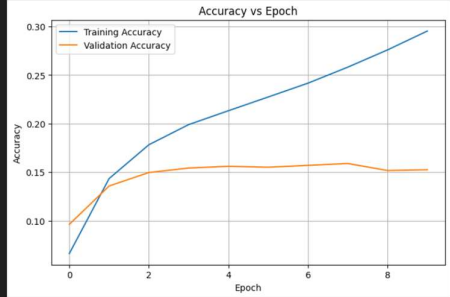
Accuracy vs Epoch



| Epoch | Training Accuracy | Validation Accuracy |
|-------|-------------------|---------------------|
| 1     | 0.0663            | 0.1434              |
| 2     | 0.1434            | 0.1525              |
| 3     | 0.2953            | 0.1525              |

```
plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Accuracy vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

Accuracy vs Epoch



| Epoch | Training Accuracy | Validation Accuracy |
|-------|-------------------|---------------------|
| 1     | 0.0663            | 0.1434              |
| 2     | 0.1434            | 0.1525              |
| 3     | 0.2953            | 0.1525              |

24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb

Generate

Code

Markdown

Run All

Select Kernel

```

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Loss vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

```

Python

| Epoch | Training Loss | Validation Loss |
|-------|---------------|-----------------|
| 0     | 6.3           | 6.3             |
| 1     | 5.8           | 6.0             |
| 2     | 5.2           | 5.9             |
| 3     | 4.8           | 5.9             |
| 4     | 4.4           | 6.0             |
| 5     | 4.1           | 6.1             |
| 6     | 3.9           | 6.2             |
| 7     | 3.8           | 6.3             |
| 8     | 3.7           | 6.4             |
| 9     | 3.6           | 6.5             |
| 10    | 3.5           | 6.6             |

24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb

Generate

Code

Markdown

Run All

Select Kernel

```

# Summary table of final performance

print("FINAL PERFORMANCE")
print("-" * 35)
print(f"Training Accuracy : {final_train_accuracy:.4f}")
print(f"Validation Accuracy : {final_val_accuracy:.4f}")
print(f"Training Loss : {final_train_loss:.4f}")
print(f"Validation Loss : {final_val_loss:.4f}")

```

Python

```

... FINAL PERFORMANCE
-----
Training Accuracy : 0.2953
Validation Accuracy : 0.1525
Training Loss : 3.7503
Validation Loss : 6.3996

```

**Screenshot checkpoint — Part C:** Capture the training output, final metrics, Accuracy vs Epoch graph, and Loss vs Epoch graph.

## PART D — Text Generation

The trained RNN predicts one word at a time. Each predicted word is

24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb

Generate

Code

Markdown

Run All

Select Kernel

## PART D — Text Generation

The trained RNN predicts one word at a time. Each predicted word is appended to the current sequence and the process is repeated to generate a complete text sample.

```

# PART D: Reverse vocabulary for converting predicted indices into
index_to_word = {index: word for word, index in tokenizer.word_index.items()}

def generate_text(seed_text, next_words=30, temperature=0.8):
    """Generate text using the trained RNN."""
    generated = seed_text.lower()

    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([generated])[0]
        token_list = pad_sequences([token_list],
                                   maxlen=SEQ_LENGTH,
                                   padding="pre",
                                   truncating="pre")

        predictions = model.predict(token_list, verbose=0)[0]

```

Python

```

predictions = model.predict(token_list, verbose=0)[0]

# Temperature controls randomness.
predictions = np.asarray(predictions).astype("float64")
predictions = np.log(predictions + 1e-8) / temperature
probabilities = np.exp(predictions) / np.sum(np.exp(predictions))

predicted_id = np.random.choice(len(probabilities), p=probabilities)

next_word = index_to_word.get(predicted_id, "")
if not next_word or next_word == "<OOV>":
    continue

generated += " " + next_word

return generated

```

Python

```

# Generate multiple samples using different seed inputs

seed_inputs = [
    "to be or not",
    "the king is",
    "my lord and",

```

Python

The image displays two screenshots of a Jupyter Notebook interface, likely Google Colab, showing the results of an RNN text generation experiment. The notebook is titled "24BAD088\_Pradeep\_Manikandan\_D\_Lab\_5.ipynb".

The top screenshot shows the output of a cell where a seed text "to be or not" is used to generate a sequence of words. The generated text is: "to be or not ye i hese i s ra sform'd o him our words presage heed or blows o chari y". Below this, another seed "the king is" is used to generate: "the king is a poor fellow a d he ha dle who by he loss of mi e ca gripe i le i his well a d i a s ra e wi h for". A third seed "my lord and" is used to generate: "my lord and is a e of her a d deed bo h a d i for a di g ' is i mi e ha have bee".

The bottom screenshot shows the output of a cell where a seed text "to be or not" is used to generate a sequence of words. The generated text is: "to be or not ha i may break his arm'd how far i have i e der for a d by ourselves u il his fierce ha res o me may ouch o ge hy fa her a d he res are you". Below this, another seed "the king is" is used to generate: "the king is bu a d all a ic were o her sorrow were a loa hs a d heir dug". A third seed "my lord and" is used to generate: "my lord and le me live a d bei g e gla d's fa her reaso ha o heir eyes warwick as he pe i g by his bu is o hi g i my embassy our a baw d".

[https://github.com/Pradeep-Manikandan-1-D/24BAD088\\_DEEP\\_LEARNING\\_LAB/upload/main](https://github.com/Pradeep-Manikandan-1-D/24BAD088_DEEP_LEARNING_LAB/upload/main)

## POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

### 1. Describe the role of the hidden state in an RNN during sequence processing.

The hidden state in an RNN acts as the model's memory during sequence processing. It stores information from previous time steps and passes relevant information to the next step. At each time step, the hidden state is updated using the current input and the previous hidden state. This allows the RNN to learn relationships and dependencies between elements in a sequence, making it useful for processing text and other sequential data.

### 2. Discuss the effect of training an RNN model for an insufficient number of epochs.

Training an RNN for an insufficient number of epochs can result in underfitting. The model may not receive enough training iterations to learn important patterns, relationships, and dependencies present in the data. Consequently, it may produce poor predictions and generate repetitive or meaningless text. The training and validation accuracy may remain low, while the loss remains relatively high. Increasing the number of epochs appropriately can help the model learn more effectively.

### **3. Explain the purpose of padding when preparing text sequences for RNN training.**

Padding is used to make text sequences equal in length before providing them to an RNN. Since sentences or sequences can contain different numbers of tokens, padding adds special values, usually zeros, to shorter sequences. This creates a uniform input shape that can be processed efficiently in batches. Padding ensures that all sequences have the same length while allowing the model to distinguish actual text tokens from added padding values.

### **4. Discuss one limitation of a Simple RNN when processing long text sequences.**

One major limitation of a Simple RNN is its difficulty in learning long-term dependencies. When processing long sequences, information from earlier time steps can gradually become weaker as it passes through many recurrent operations. This problem, known as the vanishing gradient problem, makes it difficult for the model to remember important information from the beginning of a long text. Therefore, Simple RNNs often perform better on shorter sequences.

### **5. Discuss the effect of sequence length on the performance of the text generation model.**

Sequence length determines how much previous text the RNN considers when predicting the next token. A short sequence may not provide enough context, resulting in less meaningful or grammatically incorrect generated text. A longer sequence provides more context and can improve predictions, but it also increases computational requirements and may make training more difficult. Therefore, an appropriate sequence length must balance contextual information, computational efficiency, and model performance.

## **ASSESSMENT**

| <b>Description</b>       | <b>Max Marks</b> | <b>Marks Awarded</b> |
|--------------------------|------------------|----------------------|
| Pre Lab Exercise         | <b>5</b>         |                      |
| In Lab Exercise          | <b>10</b>        |                      |
| Post Lab Exercise        | <b>5</b>         |                      |
| Viva                     | <b>10</b>        |                      |
| <b>Total</b>             | <b>30</b>        |                      |
| <b>Faculty Signature</b> |                  |                      |

